# Reporte de Práctica 4.1.1-A

## Driver de software para un relevador

*Arduino UNO R4 WiFi + Puente H L298N + LED activo en bajo*

Unidad 4

## Objetivo

Modelar por software un relevador mediante una clase en C++ que encapsule su estado, sus órdenes y sus reglas de seguridad. La clase debe poder controlar actuadores reales sin importar si se activan con nivel alto o con nivel bajo.

## Competencias desarrolladas

- Abstraer el comportamiento de un dispositivo físico en un modelo de software con estados, órdenes y restricciones.
- Aplicar encapsulamiento en C++ para proteger el hardware de órdenes incorrectas.
- Distinguir la lógica «activo en alto» de la lógica «activo en bajo» y resolverla dentro del software.
- Reconocer la necesidad de separar la parte de señal (Arduino) de la parte de potencia (fuente del motor).

## Introducción

Un pin del Arduino UNO R4 WiFi entrega apenas unos 8 mA, mientras que un motor pequeño puede pedir cientos de miliamperes al arrancar. Por eso el Arduino solo da la orden: la energía proviene de otra fuente y un dispositivo intermedio (relevador o puente H) conmuta la carga.

En lugar de repartir pines sueltos por todo el programa, se construye un modelo de software del dispositivo, y el resto del programa le habla con órdenes que tienen significado: encender y apagar. El modelo se encarga de recordar el estado del actuador, traducir la orden al nivel de pin correcto (activo en alto o en bajo), hacer cumplir las reglas físicas (un relevador real tiene partes mecánicas que se desgastan y no debe conmutarse demasiado rápido) y garantizar un estado seguro al encender o reiniciar el sistema, en el que la carga queda apagada.

Como no se cuenta con un módulo de relevadores, se utilizaron dos actuadores que lo representan: un canal del puente H L298N con un motorreductor, que actúa como «relevador electrónico» (solo enciende y apaga, sin velocidad ni cambio de sentido) y se activa con nivel alto; y un LED conectado «al revés», que imita al optoacoplador de un módulo de relevador real y se activa con nivel bajo. La misma clase controla ambos.

## Material utilizado

| Cantidad | Componente |
|---|---|
| 1 | Arduino UNO R4 WiFi y cable USB-C |
| 1 | Módulo puente H L298N |

| Cantidad | Componente |
|---|---|
| 1 | Motorreductor DC (tipo TT, 3–6 V) |
| 2 | Pilas 18650 con portapilas en serie (≈ 7.4 V) |
| 1 | LED (cualquier color) |
| 1 | Resistencia de 220 Ω |
| 1 | Protoboard |
| — | Cables jumper macho-macho y macho-hembra |

# Conexiones

## Parte de potencia (motor)

- Positivo del portapilas → terminal +12V del L298N.
- Negativo del portapilas → terminal GND del L298N.
- GND del L298N → GND del Arduino (tierra común, obligatoria).
- Motorreductor → OUT1 y OUT2.
- Se dejan puestos el jumper de ENA y el jumper 5V-EN.

## Parte de señal

| Conexión | Descripción |
|---|---|
| D7 → IN1 (L298N) | Señal de control del motor (activo en alto) |
| IN2 (L298N) → GND | Fija un solo sentido de giro: el canal solo enciende y apaga |
| 5V (Arduino) → ánodo del LED | Pata larga del LED |
| Cátodo → 220 Ω → D6 | LED activo en bajo: enciende cuando D6 está en LOW |

## Precauciones

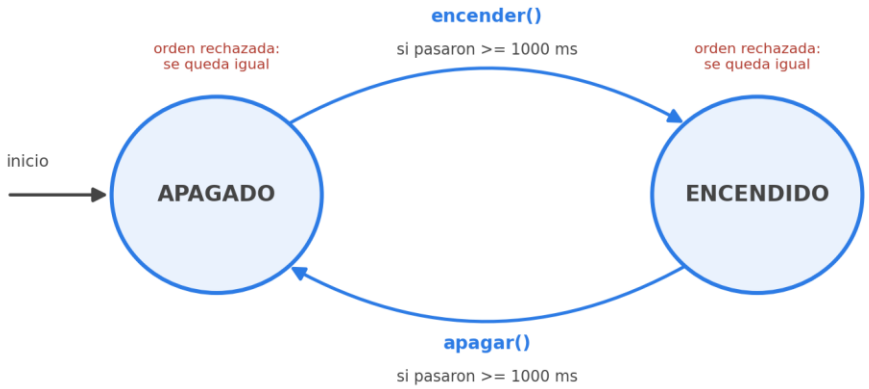
- Nunca se alimenta el motor desde el pin 5V del Arduino.
- No se conecta la terminal +5V del L298N al Arduino.
- Se revisa la polaridad del portapilas antes de conectar.
- Todo se arma con el USB desconectado.

# Diseño del modelo

## Diagrama de estados

Antes de programar se diseñó el comportamiento del relevador como una máquina de dos estados. El estado inicial es siempre APAGADO (tanto al energizar como al reiniciar), y el cambio entre estados solo

se concede si han transcurrido al menos 1000 ms desde el último cambio; de lo contrario la orden se rechaza y el pin permanece sin cambios.




## Especificación de la clase Relevador

| Elemento | Descripción |
|---|---|
| pin (privado) | Pin del Arduino que controla |
| activoEnBajo (privado) | Indica si el actuador se activa con LOW (true) o con HIGH (false) |
| encendido (privado) | Estado actual del actuador |
| ultimoCambio (privado) | Momento del último cambio, medido con millis() |
| tiempoMinimo (privado) | Tiempo mínimo entre cambios: 1000 ms |
| iniciar() | Configura el pin como salida y deja el actuador apagado |
| encender() | Enciende solo si ya pasó el tiempo mínimo; regresa true si lo logró y false si no |
| apagar() | Apaga solo si ya pasó el tiempo mínimo; regresa true si lo logró y false si no |
| estaEncendido() | Informa el estado actual |

## Reglas obligatorias

- El programa principal nunca usa digitalWrite() directamente: todo pasa por la clase.
- La clase decide si escribe HIGH o LOW según sea activo en alto o en bajo.
- Si una orden se rechaza, el pin no cambia.
- No se utiliza delay().

# Código

El programa define la clase Relevador y crea dos objetos de ella: uno para el motor (D7, activo en alto) y otro para el LED (D6, activo en bajo). La traducción de «encender/apagar» al nivel de pin se resuelve en un solo punto de la clase mediante la comparación (encender != activoEnBajo). Si el actuador ya se encuentra en el estado solicitado, la orden se considera cumplida sin escribir en el pin ni reiniciar el temporizador. Los comandos del Monitor Serie (M1, M0, L1, L0 y E) se leen carácter por carácter, de forma no bloqueante, por lo que funcionan con cualquier configuración de fin de línea.

```
/*
  Practica 4.1.1-A  Driver de software para un relevador
  Placa: Arduino UNO R4 WiFi

  Una sola clase controla dos actuadores con logica opuesta:
    Motor por L298N (D7) -> activo en ALTO
    LED al reves  (D6)   -> activo en BAJO

  Comandos del Monitor Serie (9600, con "Nueva linea"):
    M1 / M0  encender o apagar el motor
    L1 / L0  encender o apagar el LED
    E        mostrar el estado de ambos

  Sin delay(). El programa principal nunca llama digitalWrite().
*/

class Relevador {
  public:
    Relevador(uint8 t pin, bool activoEnBajo, uint32 t tMinimo = 1000)
      : _pin(pin),
        _activoEnBajo(activoEnBajo),
        _tMinimo(tMinimo),
        _encendido(false),
         ultimoCambio(0) {}

    void iniciar() {
      // Se escribe el nivel de APAGADO antes de volver el pin salida.
      // Si no, el pin arranca en LOW y un actuador activo en bajo
      // daria un pulso de encendido durante el arranque.
      digitalWrite( pin,  nivelPara(false));
      pinMode(_pin, OUTPUT);
      digitalWrite(_pin, _nivelPara(false));

       encendido    = false;
      _ultimoCambio = millis();
    }

    bool encender() { return _cambiarA(true);  }
    bool apagar()   { return _cambiarA(false); }

    bool estaEncendido() const { return _encendido; }

    // Cuanto falta para poder conmutar otra vez
    uint32_t faltaMs() const {
      uint32_t t = millis() - _ultimoCambio;
      return (t >=  tMinimo) ? 0 : ( tMinimo - t);
    }

  private:
    const uint8_t  _pin;
    const bool      activoEnBajo;
    const uint32_t _tMinimo;

    bool     _encendido;
    uint32_t _ultimoCambio;

    // Aqui vive toda la traduccion activo-alto / activo-bajo
```

```
    uint8 t  nivelPara(bool encendido) const {
      return (encendido != _activoEnBajo) ? HIGH : LOW;
    }

    bool _cambiarA(bool destino) {
      if (_encendido == destino) return true;   // ya esta ahi, no conmuta

      if (millis() - _ultimoCambio < _tMinimo) return false;  // rechazada

      digitalWrite(_pin, _nivelPara(destino));
      _encendido    = destino;
      _ultimoCambio = millis();
      return true;
    }
};

// ---------------------------------------------------

Relevador motor(7, false);   // L298N IN1 - activo en ALTO
Relevador led  (6, true);    // LED al reves - activo en BAJO

char    linea[8];
uint8_t n = 0;

// ---------------------------------------------------

void intentar(Relevador &r, bool encender, const char *nombre) {
  bool ok = encender ? r.encender() : r.apagar();

  if (ok) {
    Serial.print(nombre);
    Serial.println(encender ? " ENCENDIDO" : " APAGADO");
  } else {
    Serial.print("Orden rechazada: espere ");
    Serial.print(r.faltaMs());
    Serial.println(" ms entre cambios");
  }
}

void mostrarEstado() {
  Serial.println(F("--- ESTADO ---"));
  Serial.print(F("Motor: "));
  Serial.println(motor.estaEncendido() ? "ENCENDIDO" : "APAGADO");
  Serial.print(F("LED:   "));
  Serial.println(led.estaEncendido() ? "ENCENDIDO" : "APAGADO");
  Serial.println();
}

void procesar(const char *cmd) {
  if      (!strcmp(cmd, "M1")) intentar(motor, true,  "Motor");
  else if (!strcmp(cmd, "M0")) intentar(motor, false, "Motor");
  else if (!strcmp(cmd, "L1")) intentar(led,   true,  "LED");
  else if (!strcmp(cmd, "L0")) intentar(led,   false, "LED");
  else if (!strcmp(cmd, "E" )) mostrarEstado();
  else {
    Serial.print(F("Comando desconocido: "));
    Serial.println(cmd);
  }
}

void leerSerie() {
  while (Serial.available()) {
    char c = Serial.read();

    if (c == '\n' || c == '\r') {
      if (n > 0) {
        linea[n] = '\0';
        procesar(linea);
        n = 0;
```

```
      }
    }
    else if (n < sizeof(linea) - 1) {
      linea[n++] = toupper(c);
    }
  }
}

// ------------------------------------------------

void setup() {
  Serial.begin(9600);
  while (!Serial && millis() < 3000);

  motor.iniciar();      // estado seguro: ambos apagados
  led.iniciar();

  Serial.println(F("Driver de relevador listo"));
  Serial.println(F("Comandos: M1 M0 L1 L0 E"));
  Serial.println();
}

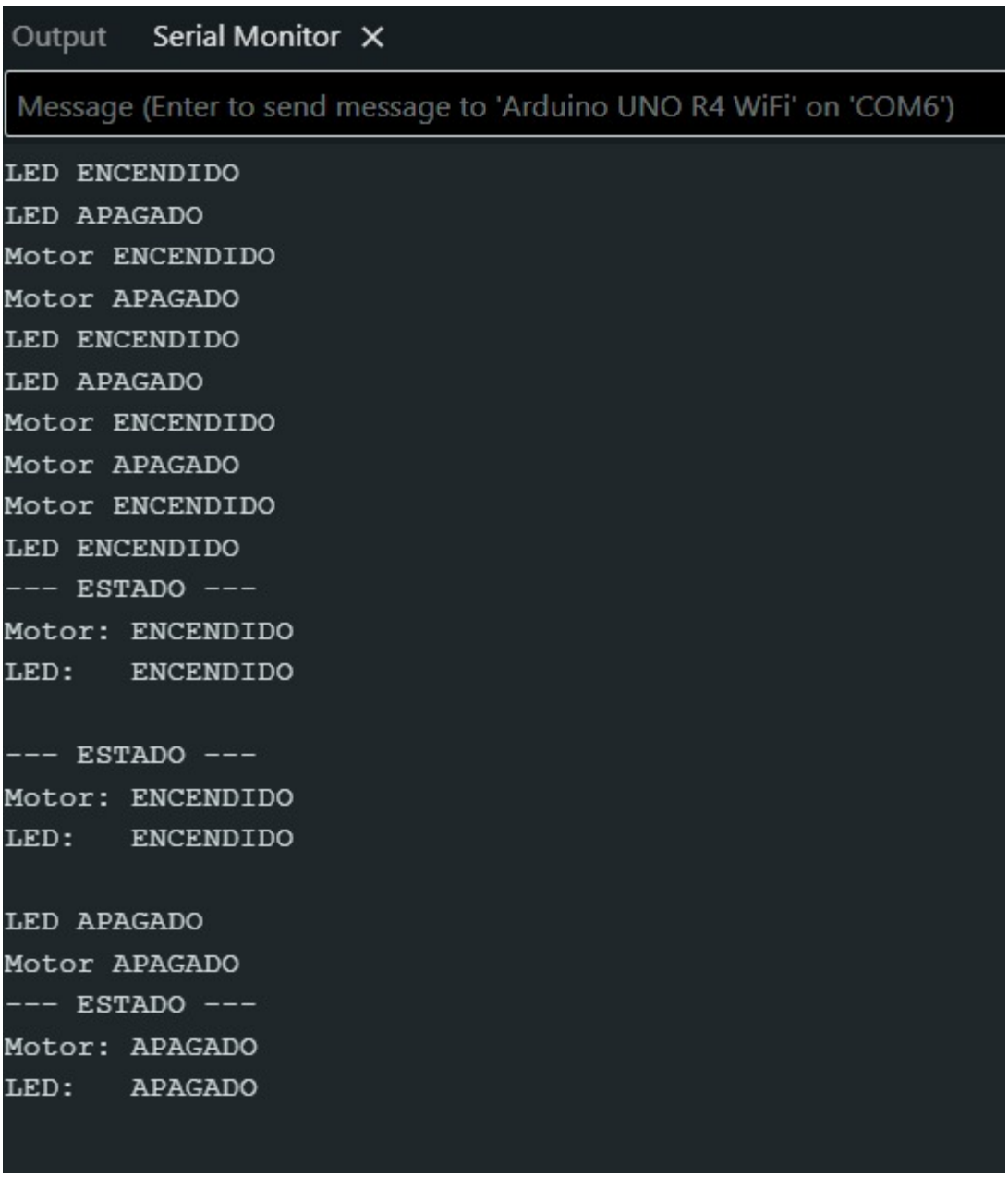
void loop() {
  leerSerie();          // unica tarea, nunca bloquea
}
```

## Pruebas y resultados

La lógica de la clase se verificó previamente compilando el código en una computadora con funciones simuladas de Arduino (pinMode, digitalWrite y millis) y comprobando el nivel que se escribe en cada pin. En esa simulación el estado inicial dejó D7 en LOW (motor apagado) y D6 en HIGH (LED apagado); la orden M1 puso D7 en HIGH; una orden M0 enviada 200 ms después fue rechazada y D7 no cambió; la misma orden enviada a los 1100 ms fue aceptada; y L1 puso D6 en LOW, encendiendo el LED. A continuación se resume el comportamiento que deben mostrar las pruebas indicadas en la práctica sobre el circuito armado.

| Prueba | Procedimiento | Resultado esperado |
|---|---|---|
| Funcionamiento normal | Enviar M1, esperar más de 1 s, enviar M0; repetir con L1 y L0; enviar E. | Cada orden responde con su mensaje, el motor y el LED cambian de estado y E informa el estado de ambos. |
| Seguridad 1: órdenes rápidas | Enviar M1 y M0 lo más rápido posible. | M1 se acepta; M0 se rechaza con «Orden rechazada: espere 1 s entre cambios» y el motor sigue encendido. |
| Seguridad 2: reinicio | Encender el motor y presionar RESET. | Al reiniciar, iniciar() deja el motor y el LED apagados. |
| Inversión | Crear el objeto del LED como «activo en alto» por error. | Ver la explicación a continuación. |

## Salida en el Monitor Serie




# Preguntas de reflexión

**1. ¿Por qué el LED se enciende con LOW y no con HIGH? Explica el recorrido de la corriente.**

El ánodo del LED está conectado a 5V y su cátodo, a través de la resistencia de 220 Ω, al pin D6. La corriente solo circula si existe una diferencia de potencial entre ambos extremos del circuito. Cuando D6 está en LOW (0 V), quedan 5 V aplicados a la serie LED + resistencia, y la corriente recorre el camino 5V → LED (ánodo a cátodo) → resistencia → pin D6, donde el pin la absorbe hacia tierra. Cuando D6 está en HIGH (≈5 V), ambos extremos tienen prácticamente el mismo potencial, no hay diferencia de tensión y no circula corriente, por lo que el LED permanece apagado. Es la misma configuración del LED de entrada de un

optoacoplador en un módulo de relevador real. Como referencia, con una caída típica de unos 2 V en el LED, la corriente estimada es de (5 − 2) / 220 ≈ 14 mA, superior a los ≈8 mA que la introducción de la práctica indica para un pin del R4; con una resistencia mayor (por ejemplo 470 Ω) la corriente sería menor, a cambio de un LED más tenue.

### 2. ¿Qué ventaja tiene que la regla del tiempo mínimo viva dentro de la clase y no en el loop()?

Al vivir dentro de la clase, la regla se cumple siempre, sin importar desde dónde se dé la orden (Monitor Serie, un botón, un sensor o cualquier otra parte del programa), porque la única forma de mover el pin es pasar por encender() o apagar(). Si estuviera en el loop(), cada lugar que controlara el actuador tendría que repetir la verificación y bastaría con olvidarla en uno solo para dañar el hardware. Además, la regla se escribe una sola vez, se reutiliza para todos los objetos de la clase, el loop() queda más limpio y una modificación posterior (por ejemplo, cambiar 1000 ms por otro valor) se hace en un único sitio.

### 3. ¿Por qué un relevador real no sirve para controlar velocidad con PWM?

El PWM controla la velocidad conmutando la carga cientos o miles de veces por segundo. Un relevador es un dispositivo electromecánico: su contacto tarda varios milisegundos en cerrarse y abrirse y no puede seguir esa frecuencia. Además, cada conmutación desgasta los contactos (chispas, arcos y picos de corriente) y su vida útil está limitada a un número finito de operaciones, que se agotaría en poco tiempo con PWM. Para variar velocidad se usan elementos de conmutación electrónicos, como transistores, MOSFET o un puente H, que sí soportan conmutaciones rápidas; el relevador queda reservado para operaciones de encendido y apagado poco frecuentes.

### 4. ¿Por qué el estado seguro es «apagado»? Da un ejemplo industrial donde convenga lo contrario (contacto NC).

Al energizar, reiniciar o ante una falla, el programa aún no sabe en qué condición se encuentra la máquina o el proceso. Si los actuadores arrancaran por sí solos podrían poner en riesgo a las personas o dañar el equipo, por lo que lo más prudente es que partan sin energía y solo se activen por una orden explícita y validada.

Hay casos donde conviene lo contrario, y para eso se usa un contacto normalmente cerrado (NC). Un ejemplo es una válvula de agua de enfriamiento de un proceso que genera calor: si se pierde la energía o falla el control, la válvula debe permanecer abierta para que el equipo siga enfriándose y no se sobrecaliente. Otro ejemplo son los frenos electromecánicos de un motor o un elevador, que se mantienen aplicados cuando no hay energía y solo se liberan al energizarlos, de modo que una falla eléctrica deja la máquina frenada. En ambos casos el estado seguro es «sin energía», aunque eso implique que la carga quede activa o frenada.

### 5. ¿Por qué es obligatoria la tierra común entre el portapilas y el Arduino?

La señal digital que sale de D7 es una tensión medida con respecto al GND del Arduino. El L298N interpreta su entrada IN1 con respecto a su propio GND; si ambos GND no están unidos, no comparten referencia, el nivel «HIGH» enviado por el Arduino no tiene un significado definido para el módulo y no existe un camino de retorno para la corriente de la señal. El resultado sería un comportamiento errático: el motor podría activarse solo, no responder o cambiar de estado por ruido. Unir los GND del portapilas, el L298N y el Arduino establece el mismo nivel de 0 V para todos, y solo así los niveles lógicos se interpretan correctamente.

**6. Si mañana cambiaras el L298N por un módulo de relevador real, ¿qué partes del programa tendrías que modificar?**

Prácticamente ninguna. La clase Relevador y la lógica del loop() permanecen igual, porque ya resuelven la polaridad y el tiempo mínimo. Solo habría que ajustar la línea donde se crea el objeto: indicar el pin al que se conecte la entrada del módulo y si es activo en alto o en bajo (muchos módulos con optoacoplador son activos en bajo, caso que la clase ya contempla). Opcionalmente podría aumentarse el tiempo mínimo entre cambios si el fabricante del relevador lo recomienda. Los cambios importantes serían de cableado: alimentar el módulo, conectar la carga a los contactos COM/NO/NC y retirar las conexiones propias del L298N (IN2 a GND, OUT1 y OUT2).

# Conclusiones

1. Modelar el relevador como una clase permitió concentrar en un solo lugar el estado del actuador, la traducción de la orden al nivel de pin y las reglas de seguridad, de modo que el programa principal solo usa órdenes con significado (encender, apagar).
2. Resolver dentro de la clase la lógica activo en alto / activo en bajo permitió controlar con el mismo código un motor y un LED que se activan con niveles opuestos, y hace que el cambio de un actuador por otro se limite a modificar los parámetros del objeto.
3. La regla del tiempo mínimo, implementada con millis() y sin delay(), protege al actuador de conmutaciones demasiado rápidas sin impedir que el programa siga atendiendo el Monitor Serie.
4. Definir el apagado como estado inicial en iniciar() garantiza que la carga quede sin energía al arrancar o reiniciar el sistema; hay que considerar, además, que entre el reinicio y la ejecución de setup() los pines están en alto impedancia, por lo que en un diseño definitivo convendría reforzar el estado seguro con una resistencia de pull-down en la entrada IN1.
5. La práctica reforzó la separación entre la parte de señal (Arduino) y la de potencia (portapilas y L298N), así como la necesidad de una tierra común entre ambas.